

Computergestützte Physik 2 | SS 2025
Übung 9 | 03.06.2025
Abgabe: 17.06.2025, 16 Uhr

Vorbereitung zum 4. E-Test am 17.06.2025

Umfang des 4. E-Tests:

- Inhalte der Vorlesungen vom 26.05.2025 bis einschließlich 02.06.2025.
- Inhalte der Übungen 8-9.
- Code von **Aufgabe 1** aus dieser Übung (= Übung 9).
- Verständnisfragen des E-Tests beziehen sich auch auf **Aufgabe 2** aus dieser Übung.
- Regeln für den E-Test sind identisch zum vorherigen E-Test.

Hinweise zur Abgabe (bitte sorgfältig lesen!):

- Die Abgabe dieser Übung ist **freiwillig** und wird keine Punkte erzielen.
- Verwenden Sie das übliche Abgaben-Format.
- Sollten Sie sich nicht an dieses vorgegebene Schema halten, wird Ihre Abgabe nicht korrigiert.
- Die Skripte müssen mit aussagekräftigen print-Statements oder deutlich gekennzeichneten Plots eine eindeutige Zuordnung der Ergebnisse zu den jeweiligen Fragestellung ermöglichen.
- Es müssen alle verwendeten Datenfiles mit abgegeben werden, mit Ausnahme von durch uns zur Verfügung gestellten Daten. Die Skripte müssen sofort lauffähig sein. Es dürfen keine input Befehle verwendet werden.
- Andere Lösungen der Übungsaufgaben, z.B. mit iPython-Notebooks oder mit MAPLE, MATLAB, MATHEMATICA, ... können wir nicht akzeptieren.

Aufgabe 0. (Elmer Installation)

Diese Aufgabe dient zur Vorbereitung auf die kommenden Übungen und ist nicht Teil dieses E-Tests oder dieser Hausaufgabenbewertung.

- Installieren Sie sich Elmer, siehe <https://www.elmerfem.org/blog/binaries/> oder <https://www.elmerfem.org/blog/compile/>, und Paraview, siehe <https://www.paraview.org/download/> auf Ihren Laptop.
- Stellen Sie sicher, dass der Installationspfad von Paraview zu Ihren Systemumgebungsvariablen hinzugefügt ist. Unter Windows wäre dies die Variable Path. Hier müssen Sie den Pfad zur paraview.exe-Datei hinzufügen, dieser wäre z.B.
C:\Program Files\ParaView 5.12.0\bin
Achten Sie darauf, dass der Paraview-Pfad unter bzw. nach dem Elmer-Pfad in der Path-Variablen steht. Möglicherweise müssen Sie Ihren Computer oder einzelne Programme anschließend neu starten.
- Testen Sie, ob Elmer läuft.
 - Öffnen Sie Elmer.
 - Laden Sie sich von dem Git-Repository *elmerfem-manuals* unter <https://github.com/ElmerCSC/elmerfem-manuals> alle Dateien aus dem Ordner `tutorials-GUI-files/Radiation` herunter und speichern Sie diese in einem gemeinsamen Ordner.

- Laden Sie anschließend in Elmer ein neues Projekt unter File -> Load Project... und wählen Sie diesen Ordner.
- Erzeugen Sie eine .sif-Datei unter Sif -> Generate. Hier passiert erstmal nichts. Sie können aber anschließend unter Sif -> Edit Text sehen.
- Starten Sie die Berechnung mit dem grünen doppelten Play-Button. Es sollte eine abfallende relative Änderung vs. Iterationsschritt des Verfahrens dargestellt werden. Im Solver Log sollte keine Fehlermeldung erscheinen.

d) Testen Sie nun Paraview.

- Starten Sie Paraview mit Elmer. Klicken Sie dazu auf Run -> Start Paraview. Sollte Elmer Paraview nicht starten können liegt das wahrscheinlich daran, dass der Pfad in Schritt b) nicht richtig angegeben wurde.
- Lassen Sie sich das 3-D Objekt anzeigen. Klicken Sie dazu auf das geschlossene Auge im Pipeline-Browser von case_t0001.vtu.
- Ändern Sie die Darstellung dieses Objekts, indem Sie unter View den Color Map Editor mittels Häkchen aktivieren. Dann sollte sich im rechten Bildschirm Drittel die entsprechende Schaltfläche öffnen. Ändern Sie die Color von Solid Color auf temperature. Nun sollte das innere Rechteck rot, die äußere Geometrie blau mit leichtem, nach außen dunkler werdenden Verlauf erscheinen.

Hinweis: Der Programmier Teil des 4. E-Tests wird sich auf diese Aufgabe (Aufgabe 1) beziehen.

Hinweis: Zur Selbstkontrolle. Der globale Koordinatenvektor sei gegeben durch

```
['x1', 'y1', 'phi1', 'x2', 'y2', 'phi2',
 'x3', 'y3', 'phi3', 'x4', 'y4', 'phi4',
 'x5', 'y5', 'phi5', 'x6', 'y6', 'phi6',
 'x7', 'y7', 'phi7', 'x8', 'y8', 'phi8']
```

Die Koordinaten bei Last sind gegeben durch

```
array([ 0. 0. 0. 0.09060484 0.04494409 -0.29547868
 0.19685395 0.06754585 -0.5454991 0.31613995 0.071529 -0.75006127
 0.44585546 0.06061725 -0.90916517 0.58339311 0.03853434 -1.02281082
 0.72614552 0.00900399 -1.09099821 0.87150531 -0.02425009 -1.11372734])
```

Aufgabe 1. (St. Martins Laterne)

Die kleine Katja hat eine hübsche, alte, gusseiserne Laterne gefunden und möchte diese zu St. Martin im Umzug präsentieren. Paul ist besorgt, ob der Laternenhalter dies aushält und möchte nun berechnen, wie sich der Stab unter der Belastung verhält.

Helfen Sie Paul mit einer FEM Berechnung wie in der Vorlesung demonstriert. Gehen Sie dabei von einer Gesamtlänge des Laternenstabs von

$$L_{\text{gesamt}} = 70 \text{ cm}$$

aus. Katja halte ihren Laternenstab in einem geschätzten Winkel von

$$\phi = 35^\circ.$$

Zerlegen Sie den Laternenstab in $N = 7$ gleich große Elemente (vgl. Abbildung 1). Verwen-

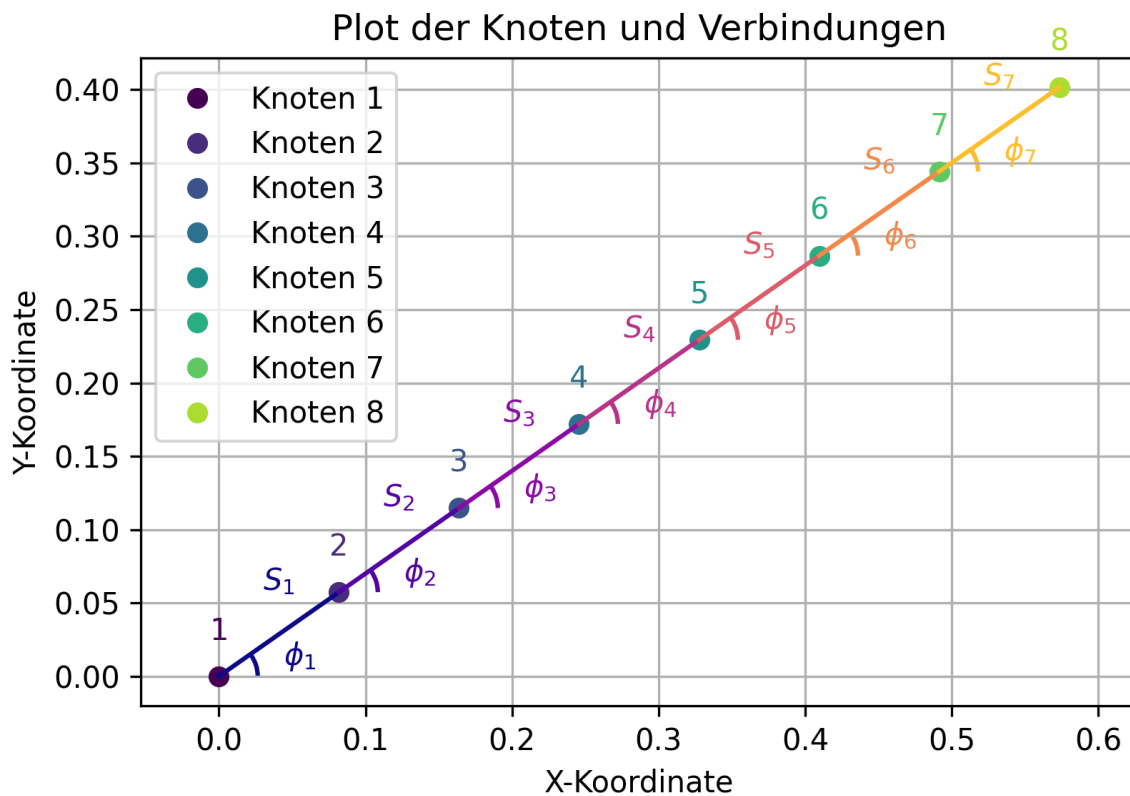


Abbildung 1: Laternenstange ohne Last

den Sie dazu die folgenden Materialeigenschaften:

$$d = 10 \text{ mm} \quad A = \pi \cdot \left(\frac{d}{2}\right)^2$$

$$E = 1800 \text{ N mm}^{-2} \quad I = \frac{\pi}{64} \cdot d^2 \cdot (0.001 \cdot d)^2 \quad \text{wobei} \quad [I] = \text{mm}^2 \cdot \text{m}^2$$

Ferner betrage die Masse $m = 0.5 \text{ kg}$. Wir vernachlässigen außerdem das Eigengewicht der Stange, berücksichtigen aber Drehmomente.

a) Übertragen Sie die Geometrie in Ihren Code:

- Definieren Sie dabei die Positionen x_i, y_i mit $[x_i, y_i] = \mathbf{m}$ der einzelnen Knoten sowie die Winkel ϕ_i in Grad der einzelnen Stangen in Abhängigkeit der oben beschriebenen Eingabengeometrie.
- Definieren Sie ein Mapping von Stangen-Id zu den linken und rechten Knoten-Ids, mit dem Sie u.a. später die globale Steifheitsmatrix bestimmen. Dieses Mapping soll vom Python-Typ `dictionary` sein. Der Aufbau sei wie folgt:

```

1         stangen_zu_knoten = {
2             StangenID: (linke_KnotenID,
3                         rechte_KnotenID),
4             ...
5         }

```

Hierbei seien alle IDs ganze Zahlen größer 0 so wie in Abbildung 1 dargestellt. Verwenden Sie die Plot-Funktion in `Library_simple_finite_elements.py` um Ihre Geometrie zu kontrollieren.

Hinweis: Stellen Sie sicher, dass Ihr Code nicht geändert werden braucht, wenn Sie Eingabewerte wie Laternenlänge, Anzahl an Elementen, etc. ändern.

b) Stellen Sie das Gleichungssystem auf:

- Verwenden Sie die beiden entsprechenden Funktionen in `Library_simple_finite_elements.py`, um eine Liste der korrekt um ϕ_i rotierten lokalen Steifheitsmatrix zu erzeugen.

Hinweis: Die ϕ_i sind hier die Winkel, um die die jeweilige Stange von ihrer horizontalen Lage rotiert wurden. Diese sind zu unterscheiden von den Winkeln θ_i , welche an jedem Knoten angeben, um wie viel dieser im Vergleich zur Stange rotiert wurde. Ohne Last sind also alle $\theta_i = 0$.

- Erzeugen Sie aus dieser Liste und Ihrem Mapping die globale Steifheitsmatrix. Achten Sie darauf, die Untermatrizen der lokalen Steifheitsmatrizen an die richtigen Stellen der globalen Steifheitsmatrix zu setzen. Verwenden Sie hierfür Ihr Mapping `stangen_zu_knoten`, sodass dieser Code-Teil eine beliebige Geometrie korrekt in eine globale Steifheitsmatrix umsetzen kann.

Hinweis: Sie können zur Kontrolle gerne die Funktion `plotMatrix` verwenden, um Ihre Matrizen anzuzeigen.

c) Lösen Sie nun das Gleichungssystem, um die Auswirkung des Gewichts auf den Laternenstab zu simulieren:

- Erzeugen Sie die reduzierte, globale Steifheitsmatrix. Löschen Sie dazu die Zeilen und Spalten Ihrer Steifheitsmatrix, die zu den Randbedingungen des Knoten 1 gehören.
- Definieren Sie den reduzierten, globalen Kraftvektor. Dieser soll nur die nach unten gerichtete Gewichtskraft der Masse m enthalten. Diese sei frei beweglich am letzten Knoten aufgehängt.
- Berechnen Sie den reduzierten Vektor der Koordinatenänderungen.
- Erzeugen Sie daraus den vollständigen Vektor der Koordinatenänderungen.
- Berechnen Sie daraus den vollständigen Kraftvektor.
- Berechnen Sie den vollständigen Koordinatenvektor bei Last.

d) Plotten Sie den Laternenstab unter Last. Nutzen Sie dafür die gleiche Funktion wie beim ersten Plot.

e) Erhöhen Sie die Anzahl der Elemente N . Wie verhält sich die Kontur des Stabs unter Last? Konvergiert sie?

Hinweis: Eine Verständnisfrage des 3. E-Tests wird sich auf diese Aufgabe (Aufgabe 2) beziehen.

Aufgabe 2. (Adaptive Schrittweiten)

In dieser Aufgabe wollen wir den Euler-Algorithmus mit adaptiver Schrittweite untersuchen. Nutzen Sie dazu Ihre Lösung der letzten Hausaufgabe (3-Körper-Problem). Vereinfachen Sie gerne die Animation, um Rechenzeit zu reduzieren.

a) Schreiben Sie eine neue Funktion `AdaptiveEulerStep_HalfStepMethod`. Diese soll den aktuellen Zustand y , Zeitpunkt t , Schrittgröße h , eine Fehlertoleranz ϵ sowie eine maximale Schrittgröße $h_{\max}$ mit Default-Wert 1 entgegennehmen. Sie gibt als Ergebnis den Zustand zum nächsten Zeitschritt y_{next} , dessen Zeitpunkt $t + h$ sowie den im nächsten Zeitschritt zu verwendende Schrittgröße h_{next} aus.

- Berechnen Sie einen Euler-Schritt vom Zustand y mit der vollen Schrittweite h .
- Führen Sie zwei Euler-Schritte vom Zustand y mit der halben Schrittweite $\frac{h}{2}$ aus.
- Vergleichen Sie nun beide Ergebnisse. Berechnen Sie dazu den lokalen Fehler e als 2-Norm der Differenz zwischen dem Zustand nach einem vollen Euler-Schritt und dem Zustand nach zwei halben Euler-Schritten.
- Wenn $e = 0$ ist, dann soll die neue Schrittgröße $2 \cdot h$ betragen. Sonst wird sie wie in der Vorlesung auf

$$h_{\text{neu}} = h \cdot \min \left(\max \left(\left(\frac{\epsilon}{e} \right)^{\frac{1}{2}} ; 0.1 \right) ; 5 \right)$$

gesetzt.

- Wenn $e \leq \epsilon$ ist, dann wird die Lösung der zwei halben Newton-Schritte zusammen mit der neuen Schrittweite h_{neu} und dem neuen Zeitpunkt $t + h$ ausgegeben.
- Sollte $e > \epsilon$ sein, dann ist die neue Schrittweite auf $\frac{h}{2}$ zu setzen und die Berechnung mit dieser neuen Schrittweite zu wiederholen. Dazu können Sie Ihre Funktion rekursiv aufrufen.

- b) Nutzen Sie die neue Funktion und plotten Sie nun alle 4 Lösungen in der Animation.
- c) Fügen Sie einen Plot zur Beobachtung der adaptiven Schrittgröße in die Animation hinzu.
- d) Was beobachten Sie im Vergleich zu den anderen Algorithmen?
- e) Wie verhält sich die adaptive Schrittweite?